MPOO 2027-1 · GRUPO 6 · PRÁCTICA 8 · MIÉRCOLES 21 DE OCTUBRE

La estación de clasificación

Herencia, segunda parte. Una banda transportadora con tres sensores y tres actuadores: primero la dibujas en un diagrama de clases y luego la programas en cuatro paquetes.

Descargar los archivos (.zip)

Qué vas a hacer

En pareja. Cuatro pasos:

1. **Dibuja el diagrama de clases**
   15 min · en papel o draw.io

2. **Escribe los cimientos: 4 clases**
   25 min · terminas cuando `PruebaTemperatura` imprime lo de la guía

3. **Escribe la estación: 7 clases más**
   45 min · terminas cuando `Main` imprime lo de la guía

4. **Entrega**
   viernes 23 de octubre, 23:59, en `p08/`

La historia

En una paquetería, los paquetes pasan por una **banda transportadora**. Tres **sensores** los vigilan: uno ve si hay un paquete atorado, otro lo pesa y otro mide la temperatura del motor. Tres **actuadores** responden: la banda, un **semáforo** y una **alarma**.

Si **cualquier** sensor da alerta, la banda se detiene, el semáforo se pone en rojo y suena la alarma. Si no, la banda avanza y el semáforo queda en verde.

Cada dispositivo trae **su propia batería**, que se gasta con el uso. La **estación** tiene hasta cinco sensores y cinco actuadores, y un sensor se puede sacar a mantenimiento y volver a instalar.

El mapa

Así queda todo cuando terminas. Tenlo a la vista en los cuatro pasos.

Quién hereda de quién, y en qué paquete vive

```
Dispositivo (abstract)        dispositivos  ◆ 1 Bateria
├── Sensor (abstract)         dispositivos
│   ├── SensorPresencia       sensores
│   ├── SensorPeso            sensores
│   └── SensorTemperatura     sensores
└── Actuador (abstract)       dispositivos
    ├── BandaTransportadora   actuadores
    ├── Semaforo              actuadores    ◆ 3 Luz
    └── Alarma                actuadores

Estacion   estacion      ◇ 1..5 Sensor y ◇ 1..5 Actuador
Bateria    dispositivos  ◆ la crea Dispositivo con new
Luz        dispositivos  ◆ las crea Semaforo con new
```

`├──` es **«es un»** (`extends`). ◆ **composición**: el dueño hace el `new` y la parte muere con él. ◇ **agregación**: la parte llega de afuera y sobrevive aunque la saquen.

Las carpetas en IntelliJ

```
src
├── dispositivos
│   ├── Bateria.java          paso 2
│   ├── Dispositivo.java      paso 2
│   ├── Sensor.java           paso 2
│   ├── Actuador.java         paso 3
│   └── Luz.java              viene en el .zip
├── sensores
│   ├── SensorTemperatura.javapaso 2
│   ├── SensorPresencia.java  paso 3
│   └── SensorPeso.java       paso 3
├── actuadores
│   ├── BandaTransportadora.javapaso 3
│   ├── Semaforo.java         paso 3
│   └── Alarma.java           paso 3
└── estacion
    ├── Estacion.java         paso 3
    ├── PruebaTemperatura.javaviene en el .zip
    └── Main.java             viene en el .zip, va al final
```

Para crear cada carpeta: clic derecho en `src` → **New → Package**. Cada archivo empieza con el `package` de su carpeta.

Paso 1 · El diagrama de clases

- **Una caja por clase del mapa** (son 12, sin `Main` ni `PruebaTemperatura`), con los atributos y métodos que salen de las descripciones de los pasos 2 y 3.
- **Los símbolos:** `+` public, `–` private, `#` protected, `~` sin modificador. Las clases y métodos `abstract`, en cursiva.
- **Las flechas, como en el mapa:** triángulo vacío hacia el padre; ◆ y ◇ del lado del dueño, con su número (`1`, `3`, `1..5`).

A los 15 minutos se proyecta la solución y corrigen el suyo.

Paso 2 · Los cimientos

Crea los cuatro paquetes y copia del .zip `Luz.java` (a `dispositivos`) y `PruebaTemperatura.java` (a `estacion`). **`Main.java` todavía no**: usa clases que aún no existen y nada correría.

Cada clase viene descrita: qué guarda, cómo se crea, qué sabe hacer y cómo se presenta. **Tú decides** los atributos, sus tipos, los modificadores y las firmas de los métodos; los nombres de los métodos y los textos entre comillas tienen que quedar tal cual, porque `Main` los usa y la salida se compara letra por letra.

1 · `Bateria` en dispositivos

La batería de cada dispositivo. Nadie fuera del paquete `dispositivos` debe poder verla ni crearla, así que la clase, su constructor y sus métodos van **sin modificador**.

- Guarda su carga, un entero de 0 a 100. Si al crearla le dan menos de 0, se queda en 0; si le dan más de 100, en 100.
- Sabe decir su carga y descargarse cierta cantidad de puntos, sin bajar nunca de 0.

2 · `Dispositivo` en dispositivos

El padre de todos. Es **abstracta**: no existe un dispositivo a secas.

- Tiene un identificador que nunca cambia y una `Bateria`.
- La batería la crea él mismo con `new` dentro de su constructor: es su **composición**.
- Se puede crear de dos formas: con id y carga inicial, o solo con id, y entonces la batería nace al 100. La segunda no repite código: llama a la primera con `this(...)`.
- Sabe dar su id (`getId`) y su carga (`getCarga`). Tiene un método `gastar` que descarga la batería y que **solo sus hijas** pueden usar.
- Se presenta (`toString`) así: `S3 (100%)`.

3 · `Sensor` en dispositivos

Un dispositivo que mide. También es **abstracta**.

- Además de lo que hereda, guarda su unidad (`C`, `kg` o `cm`), que nunca cambia, y la última señal que recibió.
- Se crea con id, unidad y carga, o solo con id y unidad (batería al 100). Uno usa `super(...)` y el otro `this(...)`.
- `recibir` guarda una señal: así `Main` simula lo que llega del mundo real. Sus hijas leen esa señal con `getSenal`, que **solo ellas** pueden usar.
- Deja dos métodos **sin cuerpo**, que cada sensor resuelve a su manera: `medir`, que da un número con decimales, y `hayAlerta`, que da verdadero o falso.
- `leer` gasta 1 de batería y devuelve un texto como `S3 mide 40.0 C`, con ` ALERTA` al final si hay alerta. **Ninguna hija puede cambiar** este método.
- Se presenta como `Sensor` seguido de lo de su padre: `Sensor S3 (100%)`.

4 · `SensorTemperatura` en sensores

Mide la temperatura del motor. Vive en otro paquete, así que necesita importar a su padre.

- Se crea con id, una calibración y una temperatura máxima. Su unidad es `C` y su batería nace al 100.
- Mide la señal más su calibración.
- Hay alerta si mide más que su máxima.
- Se presenta con lo de su padre más `  temperatura, max 45.0`.

Corre `PruebaTemperatura`:

```
package estacion;

import sensores.SensorTemperatura;

public class PruebaTemperatura {
    public static void main(String[] args) {
        SensorTemperatura termo = new SensorTemperatura("S3", -1.5,
45);
        System.out.println(termo);
        termo.recibir(41.5);
        System.out.println(termo.leer());
        termo.recibir(50);
        System.out.println(termo.leer());
        System.out.println(termo);
    }
}
```

> **DEBE SALIR**
>
> ```
> Sensor S3 (100%) temperatura, max 45.0
> S3 mide 40.0 C
> S3 mide 48.5 C ALERTA
> Sensor S3 (98%) temperatura, max 45.0
> ```

El `98%` de la última línea es la batería: cada `leer()` gastó 1.

Paso 3 · La estación

Las demás clases, en este orden.

5 · `SensorPresencia` en sensores

Ve qué tan cerca está el paquete.

- Se crea solo con su id. Su unidad es `cm`.
- La señal le llega en milímetros y él mide en centímetros.
- Hay alerta si mide menos de 2 cm: el paquete se atoró.
- Se presenta con lo de su padre más `presencia`.

6 · `SensorPeso` en sensores

La báscula.

- Guarda su límite en kilos. Si le dan un límite de 0 o menos, se queda en 1.
- Se crea de dos formas: con id, carga y límite, o con id y límite, y la batería nace al 100 (sin repetir código). Su unidad es `kg`.
- La señal le llega en gramos y él mide en kilos.
- Hay alerta si mide más que su límite.
- Se presenta con lo de su padre más `peso, limite 25.0`.

7 · `Actuador` en dispositivos

Un dispositivo que hace algo. Es **abstracta** y hermana de `Sensor`.

- Guarda si está encendido. Nace apagado.
- Se crea igual que `Dispositivo`: con id y carga, o solo con id.
- Sabe encenderse, apagarse y decir si está encendido (`encender`, `apagar`, `isEncendido`).
- Deja **sin cuerpo** el método `reaccionar`, que recibe si hay alerta y devuelve un texto con lo que hizo.
- Se presenta como `Actuador` seguido de lo de su padre.

8 · `Alarma` en actuadores

El actuador más sencillo.

- Se crea con id y volumen. El volumen se queda entre 0 y 10: si le dan 15, guarda 10.
- Al reaccionar con alerta, se enciende y devuelve `A3 suena a volumen 10`. Sin alerta, se apaga y devuelve `A3 en silencio`.
- Se presenta con lo de su padre más `alarma, volumen 10`.

9 · `BandaTransportadora` en actuadores

Mueve los paquetes.

- Se crea con id y velocidad, en metros por ciclo; si la velocidad es menor que 1, guarda 1. Lleva la cuenta de los metros que ha recorrido.
- Sabe `avanzar` de dos formas: diciéndole cuántos metros, o sin decirle nada, y entonces avanza su velocidad. La segunda forma usa la primera.
- Al avanzar: si está apagada, devuelve `A1 apagada, no avanza`. Si está encendida, suma los metros, gasta esa misma cantidad de batería y devuelve `A1 avanza 2 m, lleva 2 m`.
- Al reaccionar con alerta, se apaga y devuelve `A1 se detiene`. Sin alerta, se enciende y avanza.
- Se presenta con lo de su padre más `banda, 2 m por ciclo`.

10 · `Semaforo` en actuadores

Avisa con colores. `Luz` ya viene en el .zip.

- Tiene tres luces, `rojo`, `amarillo` y `verde`, que **crea él mismo** en su constructor y guarda en un arreglo.
- `ponerEn` enciende la luz del color que le digan y apaga las otras. `getColor` dice el color de la luz encendida, o `apagado` si ninguna lo está.
- Al reaccionar, se enciende y se pone en rojo si hay alerta, o en verde si no. Devuelve `A2 en verde`.
- Se presenta con lo de su padre más `semaforo en verde`.

11 · `Estacion` en estacion

La estación **no crea** sus dispositivos: los recibe de `Main` (**agregación**).

- Tiene un nombre que no cambia, lugar para 5 sensores y 5 actuadores (dos arreglos, cada uno con su contador) y la cuenta de ciclos.
- `instalar` sirve para un sensor y para un actuador: son **dos métodos con el mismo nombre**. Pone el dispositivo en el siguiente lugar libre e imprime `Instalado:` seguido del dispositivo. Si ya no cabe, imprime `Sin lugar para S1`.
- `retirar` recibe un id, busca ese sensor, lo saca recorriendo los que siguen un lugar a la izquierda, imprime `Retirado: S2` y **lo devuelve**. Si no lo encuentra, devuelve `null`.
- `ciclo` cuenta un ciclo más e imprime `== Ciclo 1 en Anden 3`. Luego imprime, con dos espacios al inicio, lo que lee cada sensor; si **alguno** dio alerta, hay alerta. Al final imprime, también con dos espacios, lo que hace cada actuador al reaccionar.
- Se presenta así: `Anden 3: 3 sensores, 3 actuadores, 3 ciclos`.

Con todo listo, copia `Main.java` a `estacion` y córrelo:

```
package estacion;

import actuadores.Alarma;
import actuadores.BandaTransportadora;
import actuadores.Semaforo;
import dispositivos.Sensor;
import sensores.SensorPeso;
import sensores.SensorPresencia;
import sensores.SensorTemperatura;

public class Main {
    public static void main(String[] args) {
        // los dispositivos nacen aquí, fuera de la estación
        SensorPresencia ojo = new SensorPresencia("S1");
        SensorPeso bascula = new SensorPeso("S2", 80, 25);
        SensorTemperatura termo = new SensorTemperatura("S3", -1.5,
45);
        BandaTransportadora banda = new BandaTransportadora("A1", 2);
        Semaforo semaforo = new Semaforo("A2");
        Alarma alarma = new Alarma("A3", 15);

        Estacion anden = new Estacion("Anden 3");
        anden.instalar(ojo);
        anden.instalar(bascula);
        anden.instalar(termo);
        anden.instalar(banda);
        anden.instalar(semaforo);
        anden.instalar(alarma);

        // un paquete normal
        ojo.recibir(120);
        bascula.recibir(18500);
        termo.recibir(41.5);
        anden.ciclo();

        // un paquete con sobrepeso
```

```
        ojo.recibir(95);
        bascula.recibir(32000);
        termo.recibir(43.5);
        anden.ciclo();

        // la báscula sale a mantenimiento y el motor se calienta
        Sensor fuera = anden.retirar("S2");
        ojo.recibir(150);
        termo.recibir(48);
        anden.ciclo();

        // la báscula regresa: es el mismo objeto
        System.out.println("Regresa: " + fuera);
        anden.instalar(fuera);
        banda.encender();
        System.out.println(banda.avanzar(5));
        System.out.println(anden);
    }
}
```

```
DEBE SALIR
Instalado: Sensor S1 (100%) presencia
Instalado: Sensor S2 (80%) peso, limite 25.0
Instalado: Sensor S3 (100%) temperatura, max 45.0
Instalado: Actuador A1 (100%) banda, 2 m por ciclo
Instalado: Actuador A2 (100%) semaforo en apagado
Instalado: Actuador A3 (100%) alarma, volumen 10
== Ciclo 1 en Anden 3
  S1 mide 12.0 cm
  S2 mide 18.5 kg
  S3 mide 40.0 C
  A1 avanza 2 m, lleva 2 m
  A2 en verde
  A3 en silencio
== Ciclo 2 en Anden 3
  S1 mide 9.5 cm
  S2 mide 32.0 kg ALERTA
  S3 mide 42.0 C
  A1 se detiene
  A2 en rojo
  A3 suena a volumen 10
Retirado: S2
== Ciclo 3 en Anden 3
  S1 mide 15.0 cm
  S3 mide 46.5 C ALERTA
  A1 se detiene
  A2 en rojo
  A3 suena a volumen 10
Regresa: Sensor S2 (78%) peso, limite 25.0
Instalado: Sensor S2 (78%) peso, limite 25.0
A1 avanza 5 m, lleva 7 m
Anden 3: 3 sensores, 3 actuadores, 3 ciclos
```

En el ciclo 2, **un solo** sensor en alerta detiene todo. En el ciclo 3 la estación funciona sin la báscula, y la báscula regresa con `78%`: es **el mismo objeto**, con la batería que ya había gastado.

Si algo truena

No compila

- `SensorPresencia is not abstract and does not override abstract method hayAlerta() in Sensor`: a tu sensor le falta uno de los métodos abstractos del padre. Una hija concreta tiene que escribir **todos**: `medir()` y `hayAlerta()`.
- `Sensor is abstract; cannot be instantiated`: `Sensor` es abstracta: no hay sensores a secas, solo de presencia, de peso o de temperatura.
- `call to this must be first statement in constructor`: el `this(...)`, igual que el `super(...)`, tiene que ser la primera línea del constructor.
- `senal has private access in Sensor`: la señal es privada de `Sensor`. Las hijas la leen con `getSenal()`, que es `protected`.
- `Semaforo is not public in actuadores; cannot be accessed from outside package`: a la clase le falta `public`. Sin modificador solo la ve su paquete, y `Main` vive en `estacion`.
- `Bateria is not public in dispositivos; cannot be accessed from outside package`: quisiste crear una batería desde otro paquete. La batería la crea `Dispositivo` y nadie más.
- `leer() in SensorPeso cannot override leer() in Sensor`: `leer()` es `final` en `Sensor`: las hijas no la sobrescriben, solo escriben `medir()` y `hayAlerta()`.
- `cannot find symbol`: falta un `import`. Cada clase de otro paquete se importa por su nombre completo, por ejemplo `import actuadores.Alarma;`.

Compila, pero la salida no es la de la guía

- **En el ciclo 2 la banda sigue avanzando**: tu `ciclo()` se queda con la alerta del último sensor. Lee **todos** los sensores primero y, si **alguno** dio alerta, hay alerta.
- **En el ciclo 3 vuelve a salir `S2` y falta `S3`**: tu `retirar` resta el contador pero no recorre los demás a la izquierda.
- **Sale `Regresa: null`**: tu `retirar` borra el sensor antes de guardarlo para devolverlo.
- **La alarma dice `volumen 15`**: el constructor de `Alarma` no valida.

Paso 4 · La entrega

QUÉ	DETALLE
Cuándo	**Viernes 23 de octubre, 23:59.** Sin IA, como siempre.
Con quién	En pareja (o de tres si alguien se queda solo). **Cada integrante sube todo en su propia carpeta.**
Dónde	Tu rama `entregas_apellido_nombre`, carpeta `entregas/apellido_nombre/p08/`. Solo push.
Qué	Tu carpeta `src`, la carpeta `out` (los `.class`) y una **bitácora en PDF**

La bitácora lleva: los integrantes (nombre y número de cuenta), la foto o captura de **tu diagrama**, la captura de la **salida de `Main`**, y estas tres respuestas con tus palabras:

1. **Sobrecarga contra sobrescritura:** un ejemplo de cada una sacado de tu código, y la diferencia.
2. **Composición contra agregación:** ¿por qué la `Bateria` es composición y el sensor en la `Estacion` es agregación? Piensa en quién hace el `new` y en qué le pasó a `S2` en el ciclo 3.
3. **Modificadores:** agrega en `Main` la línea `bascula.gastar(5);`, copia el error y explica por qué `SensorPeso` sí puede usar `gastar` y `Main` no. Bórrala antes de entregar.

Cómo se califica, sobre 10:

QUÉ REVISO	PUNTOS
Diagrama: las 12 cajas con sus símbolos, herencia, ◆, ◇ y multiplicidad	2
Corre y la salida es la de la guía	2
Constructores: `super(...)` en cada hija y `this(...)` en los constructores que no repiten código	2
Sobrecarga y sobrescritura: `instalar`, `avanzar` y los `@Override`	1.5
Modificadores y paquetes: atributos `private`, lo que solo usan las hijas `protected`, `Bateria` sin modificador, cada clase en su paquete del mapa	1.5
Bitácora: las tres respuestas	1

```
git checkout entregas_apellido_nombre
git pull origin main
mkdir -p entregas/apellido_nombre/p08
# copia aquí src, out y la bitácora en PDF
git add entregas/apellido_nombre/p08
git commit -m "P8: herencia, segunda parte"
git push
```

Si los `.class` no aparecen en `git status`, usa `git add -f`. El código tiene que ser suyo; si algo no sale, entréguenlo igual y cuenten en la bitácora dónde se atoraron.

Modelos de Programación Orientada a Objetos 2027-1 · Grupo 6 · Facultad de Ingeniería, UNAM

Prof. Oscar Manuel Ruiz Hurtado · Práctica 8 · Herencia, segunda parte (MADO-21, guía 8: constructores sobrecargados, `this` y `super`, sobrecarga y sobrescritura) · octubre de 2026